

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..7

ERROR HANDLING AND EXCEPTIONS

 Copy as Markdown

Error handling and exceptions

Errors on the server reach your app as typed Dart exceptions. You define an exception once, throw it on the server, and catch it by type in your Flutter app. Failures carry structured data instead of strings.

If you throw a normal exception that isn't caught by your code, it will be treated as an internal server error. The exception will be logged together with its stack trace, and a 500 HTTP status (internal server error) will be sent to the client. On the client side, this throws a `ServerpodClientException` with status code 500 (specifically `ServerpodClientInternalServerError`).

The error message and stack trace stay on the server. They are stored with the session logs in `serverpod_session_log`, not in `serverpod_log`. See [logging](#) for the difference. The Serverpod Insights app shows failed and slow calls.

Serializable exceptions

Serverpod allows adding data to an exception you throw on the server and extracting that data in the client. You define them in the same model files as any serializable model (see [Working with models](#) for how model files work). The only difference is that you use the keyword `exception` instead of `class`.

```
exception: MyException
fields:
  message: String
  errorType: MyEnum
```

Once the code is generated, you can throw that exception when processing a call to the server.

```
class ExampleEndpoint extends Endpoint {
  Future<void> doThingy(Session session) {
    // ... do stuff ...
    if (failure) {
      throw MyException(
```



```

        message: 'Failed to do thingy',
        errorType: MyEnum.thingyError,
    );
}
}
}

```

In your app, catch the exception as you would catch any exception.

```

try {
  await client.example.doThingy();
}
on MyException catch(e) {
  print(e.message);
}
catch(e) {
  print('Something else went wrong.');
```

Exception hierarchies

Related failures often share fields and the same handling in the app. Use `extends` to put them in a hierarchy, and mark the root `sealed` when you know every concrete case and want the app to handle all of them.

Define each exception in its own model file, starting with the sealed base:

api_exception.spy.yaml

```

exception: ApiException
sealed: true
fields:
  message: String

```

Then extend it with the concrete failures the endpoint can throw:

not_found_exception.spy.yaml

```

exception: NotFoundException
extends: ApiException
fields:
  resource: String

```

validation_exception.spy.yaml

```

exception: ValidationException
extends: ApiException

```

```
fields:
  field: String
```

Save with `serverpod start` running (or run `serverpod generate`), then throw the concrete exceptions from an endpoint:

```
class UserEndpoint extends Endpoint {
  Future<void> updateName(
    Session session, {
      required int userId,
      required String name,
    }) async {
    if (name.trim().isEmpty) {
      throw ValidationException(
        message: 'Enter a name.',
        field: 'name',
      );
    }

    var user = await User.db.findById(session, userId);
    if (user == null) {
      throw NotFoundException(
        message: 'The user was not found.',
        resource: 'user',
      );
    }

    await User.db.updateRow(session, user.copyWith(name: name));
  }
}
```

On the client, catch the base type and use an exhaustive switch over its concrete subtypes:

```
try {
  await client.user.updateName(userId: userId, name: name);
} on ApiException catch (error) {
  var message = switch (error) {
    NotFoundException(:final resource) => '$resource was not found.',
    ValidationException(:final field) => 'Check the $field field.',
  };

  showError(message);
}
```

Because the base is sealed, the analyzer requires the switch to cover every subtype. Adding another concrete exception then becomes a deliberate change in the app instead of a case that quietly falls through to a general catch.

A hierarchy can live in a [shared package](#) when several Serverpod projects need the same exception types. Every subtype of a sealed exception has to stay in the package that declares the base, so a consuming project

cannot add a case of its own.

Exceptions and regular model classes cannot be mixed in the same hierarchy. An exception only extends another exception, and a class only extends another class. Anything else fails validation when you generate the code.

Custom serializable exception classes

If you already have a Dart exception class in a package shared by the server and client, the class must implement `SerializableException` for Serverpod to send it to the client. Otherwise, Serverpod treats the exception as an internal server error.

```
import 'package:serverpod_serialization/serverpod_serialization.dart';

class UserFacingException implements SerializableException {
  final String message;
  final String? code;

  UserFacingException({
    required this.message,
    this.code,
  });

  factory UserFacingException.fromJson(Map<String, dynamic> json) {
    return UserFacingException(
      message: json['message'] as String,
      code: json['code'] as String?,
    );
  }

  @override
  Map<String, dynamic> toJson() {
    return {
      'message': message,
      'code': code,
    };
  }

  @override
  String toString() => message;
}
```

The class must also be known to Serverpod's generated serialization manager. Register it in the server project's `config/generator.yaml`, then run `serverpod generate`:

```
extraClasses:
  - package:my_project_shared/my_project_shared.dart:UserFacingException
```

The `SerializableException` interface marks the exception as safe to serialize to the client. See [Custom serializable classes](#) for how the `extraClasses` entry registers the type for code generation.

Default values in exceptions

Serverpod allows you to specify default values for fields in exceptions, similar to models: `default` sets the value when the field is omitted, and `defaultModel` sets it on the Dart side. See [default values](#) in Working with models.

Since exceptions are not persisted in the database, the `defaultPersist` keyword is not supported. If both `default` and `defaultModel` are specified, `defaultModel` takes precedence.

```
exception: MyException
fields:
  message: String, default="An error occurred"
  errorCode: int, default=1001
```

Handle errors in your app

A call from the client can fail in a few ways, and you usually handle each one differently:

- A **serializable exception you defined** (`MyException` above): a known, app-level failure. Catch it by its type and show the user what happened. (It is sent as an HTTP 400 response with a typed payload.)
- A `ServerpodClientHttpException`: the server answered with an error status, available as `statusCode`. Its subclasses map to HTTP status codes: `ServerpodClientBadRequest` (400), `ServerpodClientUnauthorized` (401), `ServerpodClientForbidden` (403), `ServerpodClientNotFound` (404), and `ServerpodClientInternalServerError` (500). Any other status is a `ServerpodClientUnknownHttpException`, such as the 413 returned when a call exceeds the [request size limit](#).
- A `ServerpodClientNetworkException`: the app cannot reach the server (offline, wrong URL, or a timeout). There is no status code.

Both extend the sealed `ServerpodClientException`, along with `ServerpodClientUnknownException` for failures the client cannot classify. Catching `ServerpodClientException` handles all of them at once.

A call to a [streaming method](#) that the server refuses to open fails with the same `ServerpodClientHttpException` subclasses. Failures in the stream connection itself use their own exception family. See [error handling in streams](#).

Catch the specific cases first, then fall back to the general one:

```
try {
```

```

    await client.example.doThingy();
} on MyException catch (e) {
    // A failure you defined and threw on the server.
    showError(e.message);
} on ServerpodClientUnauthorized catch (_) {
    // The call requires the user to sign in.
    redirectToSignIn();
} on ServerpodClientNetworkException catch (_) {
    // Could not reach the server.
    showError('Cannot reach the server. Check your connection and try again.');
```

```

} on ServerpodClientHttpException catch (e) {
    // The server returned an error, for example a 500.
    showError('Something went wrong (${e.statusCode}). Please try again.');
```

```

} on ServerpodClientException catch (_) {
    // Anything else the client could not classify.
    showError('Something went wrong. Please try again.');
```

```

}
```

Don't leak sensitive data

Only the serializable exceptions you define reach the client, and every field on them is sent as-is. An uncaught exception becomes a generic 500 with no details, so internal errors never leak on their own. The risk is what you put on the exceptions you do send.

- Don't put stack traces, secrets, database IDs, or internal messages into serializable exception fields. Send only what the user should see.
- Write user-facing messages, and keep the diagnostic detail in your server logs where you can look it up later.
- Validate and sanitize input before acting on it, so a bad request fails cleanly instead of surfacing an internal error.

Related

- [Database exceptions](#): the typed exceptions that database operations throw on the server.